



Pandas

Your Data's new best friend

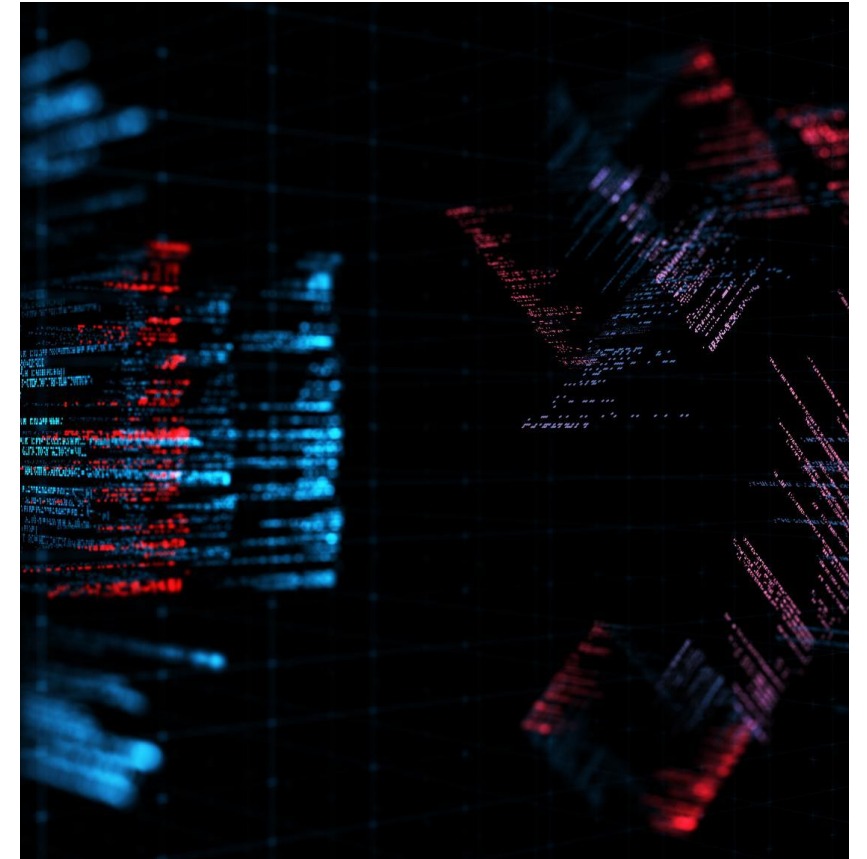
Data Structures Pandas

Tabular data

Data from tabular sources such as databases, excel, csv can be manipulated in pandas

Two Data Structures

1. **Series:** a one-dimensional labeled array holding data of any type
 - a. Can be a dictionary or a 1D Numpy array
 - b. Series has index, unlike numpy arrays. Values can be accessed by the index
2. **Dataframe:** 2-dimensional labeled data structure with columns of potentially different types.
 - a. Can be a 2D Numpy array or dictionary of 1D Numpy array, lists, tuples or series
 - b. Dataframes have index and column labels, which can be used to access elements



Creating Objects

Creating Series

- Series can be created by passing list to `pd.Series`
- Index can be customized by passing array `index=___` . Else default index is created

Creating Dataframe

- Dataframe can be created by passing list to `pd.DataFrame`
- Index and columns can be customized by passing arrays as `index=___` and `columns =` Else default index and columns are created
- Columns can be of different data types

Viewing Data

- **Viewing a sample**

`DataFrame.head()` and `DataFrame.tail()` to view the top and bottom rows of the frame respectively

- **Viewing index and columns**

`DataFrame.index` and `DataFrame.columns` to view index and columns

- **Summary of dataframe**

`describe()` gives summary statistics such as count, mean, std dev, median for numerical columns and count, unique, freq for non-numeric columns

- **Technical summary of dataframe**

`info()` gives technical details such as data types, non-null count

Accessing Data

- **Selecting column**

Column can be accessed using `df["A"]` or `df.A`

- **Selecting by label**

`Dataframe.loc` is used to select by label

- All rows matching a label can be selected by `df.loc[label]`
- All rows with select columns can be selected by `df.loc[:,column names]`

- **Selecting by position**

`Dataframe.iloc` is used to select by position

- Range of rows and columns `df.iloc[rows range, column range]`
- List of rows and columns `df.iloc[[rows list],[column list]]`

- **Filtering by condition**

`df[condition]`

Cleaning Data

- **Reindexing**

Change/add/delete the index on a specified axis

`df.reindex(index = , columns = )`

- **Drop null rows**

Drop rows that have missing data - `df.dropna()`

- **Fill null rows**

Fill rows that have missing data - `df.fillna(value=)`

Merging

- **Concatenate**

Concatenating row-wise

`pd.concat(dataframes)`

- **Merge**

SQL style join types along specific columns

- `pd.merge(left, right, how="left", on=["key1", "key2"])`
- `left.join(right, on=["key1", "key2"], how="inner")`

Exercise

- Create a House dataset with columns
 - Price
 - Number of Rooms
 - City
 - Construction year
- View summary
- Access price of 2nd and 3rd rows
- Filter by city names
- Order by price